

Technical Report: Traffic Sign Detection

Introductie

Voor mijn semesterproject heb ik ervoor gekozen om een verkeersborddetectiemodel te trainen. Ik koos dit onderwerp omdat ik graag een project rond Computer Vision wilde doen en ik online een dataset met geannoteerde data vond. Ik heb gekozen voor YOLO als model vanwege de hoge snelheid en nauwkeurigheid, dit is essentieel voor real-time detectie van verkeersborden.

GitHub-link: <https://github.com/EwoudVDE/advancedAI>

Data

Ik heb de Belgium Traffic Sign Dataset (BelgiumTS) gebruikt voor het trainen van mijn model. Deze dataset bevat annotaties die omgezet konden worden naar labels geschikt voor YOLO. Ik koos voor de reduced set, waarbij het aantal klassen verminderd wordt van 210 naar 62. Dit versnelt de training en laat me focussen op de belangrijkste verkeersborden.

Om de reduced set en YOLO te gebruiken heb ik paar aanpassingen moeten doen aan de data.

- Labels en afbeeldingen gefilterd
- IDs hernummerd van 0 t.e.m. 61
- Data opnieuw gesplitst in een 80/20 verhouding
- Afbeeldingen omgezet van .jp2 naar .jpg, omdat .jp2 niet ondersteund wordt door YOLO

Model & Methoden

Ik had de keuze uit YOLO, Vision Transformers (ViTs) en klassieke CNN's. Ik koos voor YOLO omdat het beter presteert op kleinere datasets en zeer snel inference kan uitvoeren. Dat maakt het geschikt voor real-time toepassingen zoals verkeersborddetectie.

Ik gebruikte het pre-trained model yolov11n.pt, wat het leerproces versnelt en de prestaties verbetert. Dit model is al getreind op COCO wat handig is voor kleinere datasets zoals de BelgiumTS dataset. De keuze voor het kleinste model (n voor nano) is bewust gemaakt vanwege de snelheid. Ik trainde het model gedurende 300 epochs met een patience van 30, dit betekent dat de training stopt als er 30 epochs lang geen verbetering is.

Daarnaast schakelde ik fliplr uit, omdat een gespiegeld verkeersbord mogelijk tot een andere klasse behoort.

Resultaat & Evaluatie

Het model (yolov8n.pt) leverde goede resultaten op. De algemene precisie bedraagt 0.937 en de recall 0.928, wat duidt op een degelijke classificatie van verkeersborden.

Voor sommige klassen, vooral deze met minder dan 10 afbeeldingen, waren de precisie en recall wat lager (tussen 0.5 en 0.6). Desondanks is de objectdetectie zeer accuraat: de mAP@0.5 is 0.968 en de mAP@0.5:0.95 is 0.927 — beide zijn hoge scores. Er zijn geen klassen die opvallend slechter scoren dan andere, ook al bevatten ze minder data.

De F1-curve uit de validatie toont een maximum F1-score van 0.93 bij een confidence threshold van 0.79, wat een goede balans aangeeft tussen precisie en recall.

De confusion matrix bevestigt dat het model goed presteert in het classificeren van verkeersborden. De meest voorkomende fout is het verwarren van achtergrond met een verkeersbord en omgekeerd. Dat betekent dat de classificatie heel accuraat is, maar de lokalisatie soms iets minder.

Contributie

Ik heb dit project zelfstandig uitgevoerd. Informatie over YOLO vond ik voornamelijk op de Ultralytics-website en GitHub-pagina. Voor het schrijven van scripts gebruikte ik ChatGPT als ondersteuning.

Challenges & Future Work

Het model kan nog verbeterd worden door:

- Andere hyperparameters uit te proberen (nu zijn grotendeels de standaardinstellingen gebruikt)
- Klasse-onbalans aan te pakken door meer geannoteerde data toe te voegen voor zeldzame klassen
- De volledige dataset te gebruiken in plaats van de reduced set
- Eigen klassenindeling te maken door verkeersborden te groeperen

Om de detectie verder te verbeteren kan ik achtergrondafbeeldingen zonder verkeersborden toevoegen. Ook zou ik de Ultralytics-codebase kunnen forken om klasse-gewichten of de lossfunctie aan te passen.

Een toekomstig doel zou kunnen zijn om het model live te laten draaien op een camera, en te bepalen of het herkende verkeersbord relevant is voor de bestuurder.